

Arbres de Merkle et vérification légère dans Bitcoin

Il est rappelé que le jury n'exige pas une compréhension exhaustive du texte. Vous êtes laissé(e) libre d'organiser votre présentation comme vous l'entendez. Ce texte **expose un modèle** et **énonce des résultats sans les démontrer** : les démonstrations, implémentations et expériences font l'objet de votre travail personnel. Les *suggestions de développement*, en fin de texte, ne sont là que pour vous aider ; vous ne devez pas vous sentir tenu(e) de les suivre, ni de les traiter toutes. Le jury appréciera que votre exposé s'accompagne d'exemples, d'implémentations et d'expériences menés sur ordinateur (Python conseillé : `hashlib`, `numpy`, `matplotlib`).

1 Le problème : vérifier sans tout télécharger

Une chaîne de blocs de type Bitcoin est répliquée sur des milliers de machines et pèse plusieurs centaines de gigaoctets. Un utilisateur muni d'un téléphone ne peut ni la stocker ni la revalider ; il souhaite néanmoins s'assurer qu'une transaction qui le concerne a bien été incluse dans un bloc accepté. Le problème central est celui d'une *vérification légère* : convaincre un client de faible capacité de l'appartenance d'une donnée à un très grand ensemble, à l'aide d'un *certificat* de taille minimale, sans transmettre l'ensemble ni le recalculer.

La solution retenue par Bitcoin (S. Nakamoto, 2008), devenue un standard, repose sur une structure de données *authentifiée*, l'*arbre de Merkle* (R. Merkle, 1979). Le fil conducteur de ce texte relie une brique cryptographique (le hachage), une structure arborescente et sa complexité, une garantie de sûreté fondée sur une *réduction*, la *preuve de travail* qui sécurise l'ajout des blocs, et la *politique monétaire* du protocole. Chaque notion se prête à une implémentation et à une expérimentation.

2 Anatomie d'un bloc et d'une transaction

Bitcoin identifie chaque objet par son *double SHA-256*, $H(m) = \text{SHA256}(\text{SHA256}(m)) \in \{0, 1\}^{256}$. Une transaction t a pour identifiant $\text{txid} = H(t)$. Le *modèle comptable* n'est pas fondé sur des soldes mais sur des *UTXO* (*unspent transaction outputs*) : chaque transaction consomme des sorties existantes et en crée de nouvelles, chacune verrouillée par un script. L'unité de valeur est le *satoshi*, avec $1 \text{ BTC} = 10^8 \text{ satoshis}$. Une clé publique sur la courbe **secp256k1** donne une *adresse* via $\text{HASH160}(x) = \text{RIPEMD160}(\text{SHA256}(x))$, encodée en Base58Check ; les sorties sont dépensées en fournissant une signature ECDSA valide.

Un *bloc* réunit une liste ordonnée de transactions $t_0, \dots, t_{n-1}$ et un *en-tête* de 80 octets contenant : un numéro de version, le haché de l'en-tête précédent (ce qui *chaîne* les blocs), la *racine de Merkle* des transactions, un horodatage, un champ compact **bits** codant la *cible* de difficulté, et un *nonce*. La première transaction, dite *coinbase*, crée de la monnaie neuve : elle verse au mineur une *subvention* plus la somme des frais des autres transactions.

3 L'arbre de Merkle d'un bloc

La *racine de Merkle* résume les transactions et figure dans l'en-tête. On procède par niveaux : partant des feuilles txid_i , on remplace chaque paire consécutive (a, b) par $H(a\|b)$; si un niveau contient un nombre impair de nœuds, on duplique son dernier élément avant l'appariement (règle propre à Bitcoin). On s'arrête sur un unique haché.

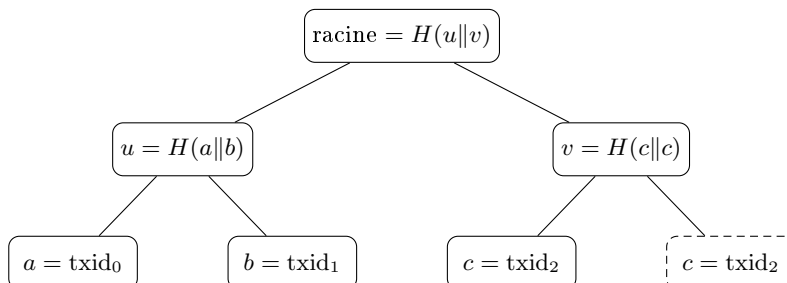


Figure 1: Arbre de Merkle d'un bloc à **trois** transactions ($\text{txid}_0, \text{txid}_1, \text{txid}_2$) : le nombre de feuilles étant impair, le dernier haché $c = \text{txid}_2$ est *dupliqué* (feuille en pointillés) afin de compléter la paire, puis $v = H(c\|c)$.

Trois propriétés, énoncées ici sans preuve, structurent la construction et sa vérification.

Proposition 1 (Terminaison et correction de la construction). *L'algorithme par niveaux termine : la longueur du niveau courant est un entier qui décroît strictement (variant $\lceil t/2 \rceil < t$ dès que $t \geq 2$) jusqu'à valoir 1. Il est correct : à chaque étape, chaque nœud est le haché de l'unique couple de ses enfants (invariant), de sorte que la valeur finale ne dépend que de la liste ordonnée des feuilles.*

Proposition 2 (Coût de construction). *Le calcul de la racine de n transactions demande $O(n)$ évaluations de H , en temps et espace $O(n)$, et l'arbre a une hauteur $\lceil \log_2 n \rceil$. La taille d'une preuve d'inclusion, et le coût de sa vérification, sont donc en $\Theta(\log n)$.*

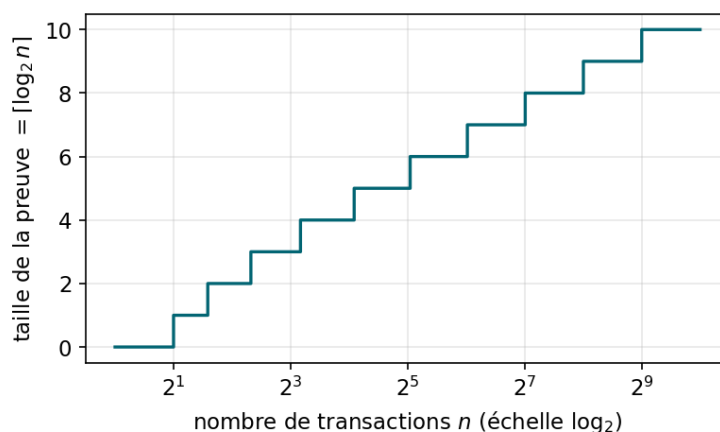


Figure 2: Allure attendue (à obtenir par simulation) : taille moyenne d'une preuve d'inclusion en fonction du nombre n de transactions ; on doit retrouver l'escalier $\lceil \log_2 n \rceil$.

Remarque 1 (Une subtilité de sécurité). La règle de duplication a une conséquence inattendue : pour certaines valeurs de n , un bloc et un bloc obtenu en dupliquant réellement des transactions peuvent produire la *même* racine (ambiguïté connue sous le nom de « CVE-2012-2459 »). Une structure authentifiée doit donc être analysée avec soin : la racine seule ne garantit pas l'unicité de l'arbre sous-jacent.

4 Hachage cryptographique et collisions

La sécurité repose entièrement sur les propriétés de H . On distingue la *résistance à la préimage*, à la *seconde préimage* et aux *collisions*. Cette dernière est cruciale ici.

Proposition 3 (Existence des collisions). *L'application $H : \{0, 1\}^* \rightarrow \{0, 1\}^{256}$ n'est pas injective : des messages distincts de même haché existent nécessairement.*

Les collisions existent donc, mais on *postule* qu'elles sont introuvables en temps raisonnable (hypothèse *calculatoire*, non un théorème). Sa force se mesure par l'attaque générique la plus simple, analysée dans le modèle de l'*oracle aléatoire*.

Proposition 4 (Borne des anniversaires). *En tirant k valeurs indépendantes et uniformes parmi $M = 2^b$, la probabilité d'absence de collision vaut $q_k = \prod_{i=0}^{k-1} (1 - i/M)$ et vérifie $q_k \leq \exp(-k(k-1)/(2M))$. Une collision devient probable dès que $k \gtrsim 1,177 \cdot 2^{b/2}$.*

Une empreinte sur b bits n'offre ainsi qu'une résistance de l'ordre de $2^{b/2}$: le choix $b = 256$ vise une marge de 2^{128} . Le cas $M = 365$ n'est autre que le *paradoxe des anniversaires*. Ces seuils s'observent par une expérience de Monte-Carlo sur une empreinte tronquée. La qualité d'une telle estimation est régie par la loi des grands nombres.

Proposition 5 (Vitesse de convergence de l'estimateur Monte-Carlo). *Un estimateur de Monte-Carlo d'une probabilité (ou d'une espérance) fondé sur N tirages indépendants a un écart-type en $\Theta(1/\sqrt{N})$: pour gagner un chiffre décimal de précision, il faut multiplier N par 100. En échelle log-log, l'erreur empirique décroît le long d'une droite de pente $-1/2$.*

```
import hashlib
def H(m: bytes) -> bytes:
    return hashlib.sha256(hashlib.sha256(m).digest()).digest()
def tronque(h: bytes, b: int) -> int: # b premiers bits
    return int.from_bytes(h, "big") >> (256 - b)
```

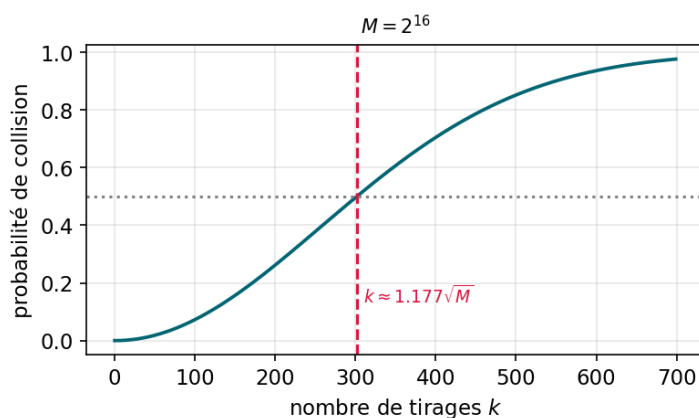


Figure 3: Allure attendue (à obtenir par simulation) : probabilité d'au moins une collision en fonction du nombre de tirages, pour une empreinte de $b = 16$ bits ; la transition se produit au voisinage de $1,177\sqrt{M} = 1,177 \cdot 2^8$.

5 Preuves d'inclusion et clients légers (SPV)

Un client *léger* (*Simplified Payment Verification*) ne conserve que les en-têtes, donc les racines. Pour lui prouver qu'une transaction t appartient à un bloc de racine R , on lui fournit une *preuve*

de Merkle : la suite des hachés *frères* rencontrés en remontant de la feuille $H(t)$ jusqu'à la racine, chacun accompagné de son côté. Le client recombine par H et accepte si et seulement s'il retrouve R . La longueur d'une telle preuve est $\lceil \log_2 n \rceil = O(\log n)$: une poignée de hachés là où le bloc entier serait prohibitif.

La sûreté de ce mécanisme se ramène à la seule hypothèse sur H .

Théorème 1 (Sûreté de la vérification, ou *soundness*). *Si H est résistante aux collisions et si la vérification d'une preuve pour une valeur x aboutit à la racine authentique R , alors — sauf à exhiber une collision de H — la valeur x figure réellement parmi les transactions ayant produit R .*

Autrement dit, forger une fausse preuve d'inclusion revient à calculer une collision : la sécurité de l'édifice se réduit à celle de H , ce qui justifie le dimensionnement à 256 bits. En pratique, un client SPV allège encore ses échanges à l'aide de *filtres de Bloom* (BIP-37) : il communique une structure probabiliste identifiant *approximativement* les transactions qui l'intéressent, acceptant de « faux positifs » pour préserver un minimum de confidentialité.

6 Preuve de travail et politique monétaire

Rester léger suppose de faire confiance à la chaîne d'en-têtes reçue ; Bitcoin la rend coûteuse à falsifier par la *preuve de travail*. L'en-tête contient un *nonce* ν ; *miner* le bloc consiste à trouver ν tel que $H(e_\nu)$, vu comme un entier sur 256 bits, soit inférieur à la cible T . Dans le modèle de l'oracle aléatoire, chaque essai réussit indépendamment avec probabilité $p = T/2^{256}$.

Proposition 6 (Coût du minage). *Le nombre N d'essais jusqu'au premier succès suit une loi géométrique de paramètre p ; son espérance est $\mathbb{E}[N] = 1/p$, et le minage termine presque sûrement.*

Le minage est donc un algorithme *probabiliste* de type Las Vegas : résultat toujours correct, temps aléatoire d'espérance $1/p$. Le protocole *ajuste* périodiquement la cible — toutes les 2016 blocs, soit environ deux semaines — de manière à maintenir un intervalle moyen de dix minutes entre blocs, malgré les variations de la puissance de calcul du réseau.

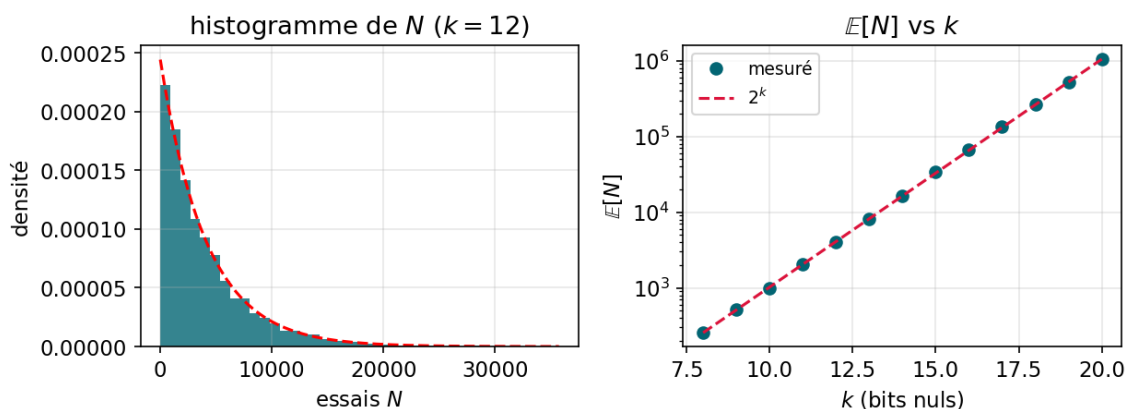


Figure 4: Allure attendue (à obtenir par simulation) : à gauche, l'histogramme du nombre d'essais N épouse une décroissance géométrique ; à droite, l'espérance mesurée $\mathbb{E}[N]$ suit la droite 2^k en échelle semi-logarithmique.

La coinbase suit, elle, une *politique monétaire* déterministe : la subvention par bloc est divisée par deux tous les 210 000 blocs (*halving*). La masse monétaire totale est donc gouvernée par une série géométrique, dont la convergence est elle-même géométrique.

Proposition 7 (Masse monétaire bornée et convergence géométrique). *Avec une subvention initiale de 50 BTC divisée par deux toutes les 210 000 blocs, l'émission totale est majorée par $210\,000 \cdot 50$.*

$\sum_{i \geq 0} 2^{-i} = 21 \cdot 10^6$ BTC. De plus, après la e -ième ère de halving, la fraction déjà émise vaut $1 - 2^{-(e+1)}$: l'écart au plafond décroît géométriquement en 2^{-e} .

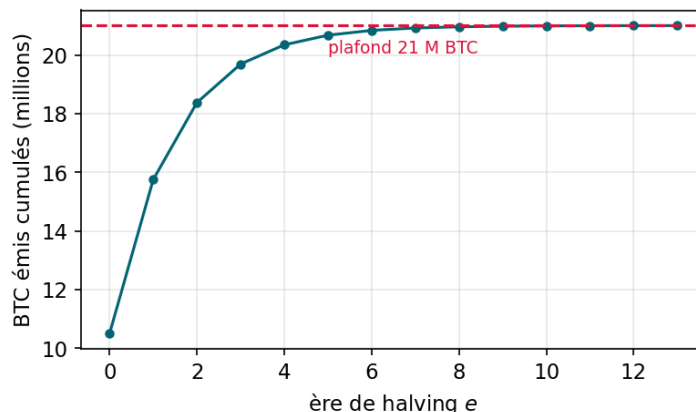


Figure 5: Allure attendue (à obtenir par simulation) : BTC cumulés émis au fil des ères de halving, convergeant géométriquement vers le plafond de 21 millions.

Enfin, la *sécurité* de la chaîne se lit comme une course : un attaquant de puissance relative $\alpha < 1/2$, parti avec z blocs de retard, ne rattrape la chaîne honnête qu'avec une probabilité décroissant exponentiellement en z . C'est le fondement de la règle des *confirmations* et le lien entre la vérification légère du client et la robustesse du consensus.

7 Structures voisines

L'arbre binaire de Bitcoin admet des variantes. Ethereum emploie un arbre de *Patricia-Merkle* pour indexer un état modifiable ; les *arbres de Verkle* remplacent le hachage par des *engagements polynomiaux* et abaissent la taille des preuves de $O(\log n)$ à $O(1)$. Plus généralement, la vérification « à faible coût » illustrée ici inspire les *preuves à divulgation nulle* et les *rollups*.

Suggestions pour le développement

Pistes largement indépendantes ; on n'en traitera qu'une partie, en privilégiant celles que l'on peut à la fois exposer, implémenter et éprouver par une simulation.

- Prouver la Proposition 1 (terminaison via le variant, correction via l'invariant) puis la Proposition 2 ; implémenter la construction, la génération et la vérification des preuves, et retrouver expérimentalement l'escalier $\lceil \log_2 n \rceil$ (Figure 2).
- Démontrer la Proposition 4, en discuter l'équivalent asymptotique et la confronter à une expérience de Monte-Carlo (Figure 3) ; établir la Proposition 5 et vérifier la pente $-1/2$ en échelle log-log.
- Prouver le Théorème 1 et expliciter la *réduction* : comment une fausse preuve fournit-elle effectivement une collision ?
- Approfondir la Remarque 1 : caractériser les tailles de blocs sensibles à l'ambiguïté de duplication et proposer une parade.
- Étudier et simuler le modèle de minage (Proposition 6) : distribution de N , effet de l'ajustement de difficulté, temps moyen entre blocs.
- Établir la Proposition 7, mettre en évidence la convergence géométrique vers 21 millions (Figure 5) et discuter la place des frais lorsque la subvention s'éteint.

- Modéliser et simuler la « course entre chaînes » : probabilité de rattrapage en fonction de α et z ; en déduire un choix du nombre de confirmations.
- Analyser les filtres de Bloom (BIP-37) : taux de faux positifs, compromis confidentialité/bande passante pour un client SPV.
- Comparer, en complexité et en taille de preuve, l'arbre binaire de Bitcoin, un arbre k -aire, l'arbre de Patricia-Merkle et les engagements de type Verkle.

Références

1. S. Nakamoto, *Bitcoin: A Peer-to-Peer Electronic Cash System*, 2008.
2. R. C. Merkle, *A digital signature based on a conventional encryption function*, CRYPTO '87.
3. A. M. Antonopoulos, *Mastering Bitcoin*, O'Reilly ; A. M. Antonopoulos, G. Wood, *Mastering Ethereum*, O'Reilly.
4. J. Katz, Y. Lindell, *Introduction to Modern Cryptography*, CRC Press.